

Candlelight Mod Repo - zmienić potem na faktyczny tytuł

Jakub Zygmunt KLIMCZAK
Id: 167314

Kierunek: Informatyka
Specjalizacja: Zaawansowane systemy baz danych

Opiekun
dr inż. Lesław Pawlaczyk

Chorzów, 2025

Spis treści

1	Wprowadzenie	3
1.1	Przedmowa	3
1.2	Inspiracja	3
1.3	Cel oraz zakres pracy	4
1.4	Konkurencyjne rozwiązania	4
2	Podstawy teoretyczne wybranych technologii	5
2.1	Używane IDE oraz ich rozszerzenia	5
2.1.1	Visual Studio 2022	6
2.1.2	Visual Studio Code	6
2.2	PostgreSQL	7
2.3	.NET oraz Entity Framework	7
2.3.1	.NET 8	8
2.3.2	EF Core 8 oraz Npgsql	8
2.4	Biblioteki backendowe	9
2.4.1	SpaProxy	9
2.4.2	Steam OpenID	9
2.5	Typescript i Angular	9
2.5.1	TypeScript	9
2.5.2	Angular	10
2.6	Znaczące biblioteki frontendowe	10
2.6.1	RxJS	10
2.6.2	Angular Material	11
2.7	Linting kodu i zapobieganie code smellowi	11
2.7.1	GitHub Actions	11
2.7.2	ESLint	11
2.7.3	Prettier	11
2.7.4	ReSharper	11
2.7.5	SonarQube	11
2.8	Biblioteki testowe	11
2.8.1	Karma Jasmine	11

2.8.2	NUnit	11
2.8.3	Moq	11
2.8.4	Coverlet	11
3	Architektura projektu	13
4	Implementacja funkcjonalności aplikacji	17
4.1	Obsługa autoryzacji użytkownika	17
4.2	Zarządzanie profilem użytkownika	17
4.3	Funkcjonalności związane z grami	17
4.4	Funkcjonalności związane z modyfikacjami	17
4.4.1	Przesyłanie modyfikacji oraz ich wersji	17
4.4.2	Lista modyfikacji dla gry	17
4.5	Strona główna oraz UX aplikacji	17
4.6	Integracje z zewnętrznymi serwisami	17
5	Podsumowanie projektu	19
5.1	Trudności i wyzwania	19
5.2	Wnioski z projektu	19
5.3	Potencjalne kierunki rozwoju projektu	19

Rozdział 1

Wprowadzenie

1.1 Przedmowa

Modyfikacje do gier stają się z biegiem lat standardem branży w sektorze PC. Wiele gier zawdzięcza dużą część swojej widowni właśnie nim. Modyfikacje, potocznie zwane 'mody', są - jak nazwa wskazuje - modyfikacjami kodu gier zazwyczaj stworzonymi przez użytkowników (a nie oryginalnych twórców) w celu zmiany zachowania bądź działania danego programu. Coraz częściej ludzie chcą dzielić się swoimi tworem z innymi pasjonatami. Często jednak ten proces jest bardziej skomplikowany, niż być powinien.

Tematem niniejszej pracy magisterskiej jest odpowiedź na tę potrzebę - aplikacja Candlelight. Jej celem jest stworzenie wygodnego, uporządkowanego i otwartego repozytorium modyfikacji do gier komputerowych. Candlelight umożliwia przeglądanie, ocenianie oraz pobieranie modyfikacji, a także pozwala twórcom publikować nowe wersje i zarządzać nimi. Aplikacja kładzie nacisk na przejrzystość interfejsu, bezpieczeństwo użytkowników oraz skalowalność systemu. System uwzględnia funkcjonalności takie jak mechanizm ulubionych, ocen i recenzji oraz obsługę kont użytkowników, w tym integrację z platformą Steam.

W kolejnych rozdziałach szczegółowo przedstawione zostaną założenia, implementacja oraz możliwości rozwoju tej platformy.

1.2 Inspiracja

Historia modyfikacji do gier sięga nawet jednej z najstarszych gier komputerowych, *Spacewar!*. Była ona wielokrotnie przerabiana oraz rozszerzana przez znajomych autora, Steve'a Russella, którzy zmieniali bądź dodawali jej zawartość [kluska2008]. Scena modyfikacji do gier niezmiennie się od tego czasu zmieniała, coraz szersze grono pasjonatów poświęca wielką ilość czasu na nieodpłatny rozwój swoich ulubionych gier komputerowych. Będąc odbiorcą ich pracy oraz użytkownikiem zarówno modyfikacji jak i wielu

stron internetowych im przeznaczonym postanowiłem spróbować swoich sił w ujednoliceniu i zuniwersalizowaniu tego rozproszonego makroświata.

1.3 Cel oraz zakres pracy

Celem niniejszej pracy jest zaprojektowanie oraz implementacja nowoczesnej aplikacji webowej pełniącej funkcję repozytorium modyfikacji do gier komputerowych. Aplikacja ma umożliwiać użytkownikom przeglądanie, wyszukiwanie, ocenianie oraz publikowanie modów, jak również zarządzanie własnym profilem i ulubionymi pozycjami. Oprócz części funkcjonalnej, istotnym elementem pracy jest także przemyślane zaprojektowanie architektury systemu, w taki sposób, aby był on łatwy w dalszym rozwoju, testowaniu i utrzymaniu.

Zakres pracy obejmuje zarówno backend, jak i frontend aplikacji, wraz z integracją z zewnętrznymi usługami (uwierzytelnianie za pomocą konta Steam), projektowaniem bazy danych oraz wdrożeniem kluczowych mechanizmów umożliwiających publikację i wersjonowanie modów.

W projekcie szczególny nacisk położyłem na stosowanie dobrych praktyk inżynierii oprogramowania oraz podejścia zwinnego (Agile). Dążyłem do tworzenia kodu czytelnego, modularnego, łatwo testowalnego oraz rozszerzalnego w miarę rozwoju wymagań. Starłem się również projektować komponenty w sposób umożliwiający ich ponowne wykorzystanie oraz izolowanie odpowiedzialności (zgodnie z zasadą Single Responsibility Principle), co jest szczególnie istotne przy budowie aplikacji o wielowarstwowej strukturze.

W efekcie, końcowy system nie tylko spełnia założenia funkcjonalne, ale także zgodnie z planem stanowi solidną bazę do dalszego rozwoju, zarówno w kontekście rozbudowy o nowe funkcjonalności przeze mnie lub innych chętnych deweloperów, jak i potencjalnego wdrożenia w środowisku produkcyjnym. Aplikacja stanowi idealną bazę dla osoby chcącej nauczyć się pracy z większym projektem, a z racji open-sourcowej licencji jest ona dostępna do wglądu bądź rozszerzenia dla wszystkich zainteresowanych.

1.4 Konkurencyjne rozwiązania

Rozdział 2

Podstawy teoretyczne wybranych technologii

W niniejszym rozdziale opiszę używane przeze mnie technologie oraz przyczyny, dla których je wybrałem. Główny stos technologiczny opiera się na rozwiązaniach firmy *Microsoft*, takich jak *ASP.NET Core* oraz *TypeScript*, co pozwoliło na stworzenie nowoczesnej, wydajnej i skalowalnej aplikacji webowej.

Pomimo dostępności systemu *Microsoft SQL Server*, jako bazę danych zdecydowałem się zastosować *PostgreSQL*, jedno z najpopularniejszych i najlepiej wspieranych rozwiązań open-source w tej kategorii. Wybór ten był podyktowany jego elastycznością, zaawansowanymi możliwościami, pełną kompatybilnością z wykorzystywanym narzędziem *Entity Framework Core* oraz chęcią zaznajomienia się z nim.

Zamiast technologii *ASP.NET WebForms*, które historycznie związane są ze środowiskiem *Microsoft*, zdecydowałem się na nowoczesne podejście z wykorzystaniem *ASP.NET Core* oraz architektury SPA (Single Page Application) opartej o *Angular*. *WebForms* są uznawane za przestarzałe i niezalecane w przypadku nowych projektów, brakuje im pełnego wsparcia dla współczesnych standardów webowych, komponentów responsywnych oraz rozdzielenia logiki od widoku, co znacząco ogranicza możliwości rozwoju i skalowalności aplikacji. Inną alternatywą *Microsoftu* byłby *Blazor*, ale *Angular* ma dużo lepsze wsparcie 3rd party, a głównym jego atutem jest biblioteka *Angular Material*.

2.1 Używane IDE oraz ich rozszerzenia

Podczas pracy nad projektem używałem dwóch IDE (Integrated Development Environment): *Visual Studio 2022* podczas pracy nad backendem, oraz *Visual Studio Code* podczas pracy nad frontendem oraz do edycji plików konfiguracyjnych i skryptów. Dodatkowo stosowałem *pgAdmin4* [pgadmin2025] do okazjonalnego bezpośredniego skryptowania bazy danych w celach debugowania bądź szybkiego podejrzenia poszczególnych

tabel.

2.1.1 Visual Studio 2022

Visual Studio jest profesjonalnym zintegrowanym środowiskiem programistycznym powszechnie używanym w branży IT. Zostało stworzone przez firmę *Microsoft*, przez co jest znakomicie przystosowane do pracy nad jej flagową platformą, *.NET*em. [vs2022docs]

Wygodę zwiększa integracja z biblioteką do testów jednostkowych, integracja z *GitHub*em oraz *GitHub Actions*, mechanizmy automatycznego refaktoryzowania kodu, zaawansowany debugger, a także zarządzanie wieloma projektami w ramach rozwiązania. *IntelliSense* wbudowany w środowisko bardzo przyspiesza analizę kodu oraz błędów bez potrzeby kompilacji, pomaga znaleźć miejsca, w których używane są dane zmienne, klasy i metody, a także podpowiada m.in. składowe klas czy argumenty metod. Jest to flagowy produkt *Microsoftu*, co sprawia, że dostaje on częste łatki oraz najwyższej jakości wsparcie.

Kolejnym atutem jest wbudowany system pakietów *NuGet* [nuget2025], dzięki któremu można łatwo zarządzać zależnościami projektu, a także *Visual Studio Marketplace* [vsm2025], które pozwala na wygodne dodawanie rozszerzeń spośród olbrzymiej ilości przygotowanej przez innych developerów.

Podczas pracy nad projektem korzystałem z rozszerzenia *ReSharper* [resharper2025], które pozwala przyspieszyć wiele akcji często wykonywanych na kodzie, jak zmiana nazw zmiennych oraz plików, a także informuje o większej ilości błędów oraz problemów niż sam *IntelliSense*.

Drugim wykorzystywanym przeze mnie rozszerzeniem jest *SonarQube*, który pokazuje, jaka część kodu jest pokryta testami, co pomaga zauważyć luki w testach jednostkowych oraz poprawić jakość i niezawodność tworzonego oprogramowania. Do tego pomaga monitorować kod pod kątem potencjalnych błędów, luk bezpieczeństwa czy niezgodności ze standardami [sonarqube2025vs].

2.1.2 Visual Studio Code

Visual Studio Code to lekkie środowisko programistyczne opracowane przez firmę *Microsoft*. W odróżnieniu od *Visual Studio*, jest to edytor kodu z rozszerzalnym ekosystemem, który po odpowiedniej konfiguracji może pełnić funkcję pełnoprawnego IDE. *Visual Studio Code* cechuje się bardzo szybkim działaniem, niskim zużyciem zasobów oraz bogatym zestawem dostępnych rozszerzeń wspierających różne języki i technologie.

Doskonale sprawdza się on zarówno do szybkich edycji plików tekstowych oraz konfiguracyjnych, jak i tworzenia komponentów w *Angularze*. Dzięki wbudowanemu terminalowi można wygodnie odpalać komendy w trakcie pisania, bez zmiany okna. Wbudowane wsparcie dla systemu *Git* oraz podpowiedzi kodu oparte na mechanizmie *IntelliSense*

[[vscodeIntellisense2025](#)] sprawiają, że nie odstaje on zbyt od dużo cięższego konkurenta, *Visual Studio*.

Wybrałem *Visual Studio Code* do pisania frontendu zamiast pisać wszystko w *Visual Studio* głównie z powodu lepszego wsparcia dla technologii webowych, takich jak *Angular*, *SCSS* oraz *HTML*, a także dzięki znacznie lżejszej i bardziej responsywnej pracy edytora przy projektach frontendowych.

2.2 PostgreSQL

PostgreSQL to obiektowo-relacyjny system zarządzania bazą danych (ORDBMS), uznawany za jedno z najbardziej zaawansowanych i stabilnych rozwiązań tego typu w świecie open-source. Jest szeroko wykorzystywany zarówno w aplikacjach webowych, jak i systemach analitycznych, dzięki rozbudowanym możliwościom przechowywania danych, obsłudze transakcji ACID, wsparciu dla indeksów złożonych, widoków materializowanych i zapytań zagnieżdżonych [[postgresql2025](#)].

W projekcie *PostgreSQL* służył jako główna baza danych zarówno dla danych użytkowników, jak i przechowywanych danych dotyczących między innymi modów, gier, ocen. Do zarządzania bazą danych i jej strukturą wykorzystywałem migracje *EF Core*, natomiast w przypadkach wymagających ręcznego podejrzenia danych, edycji czy debugowania — posługiwałem się narzędziem *pgAdmin4*.

PostgreSQL został wybrany jako system bazodanowy przede wszystkim dlatego, że należy do najpopularniejszych technologii tego typu (według zeszłorocznej ankiety [StackOverflow](#) [[stackoverflow2024survey](#)] używany przez aż 51.9% profesjonalnych developerów), co w świecie IT przekłada się na długowieczność projektu, lepsze wsparcie i dużą dostępność zasobów edukacyjnych. Jego pełna zgodność z *Entity Framework Core* dzięki stabilnemu i rozwijanemu providerowi *Npgsql* [[npgsql2025](#)] pozwoliła na płynną integrację z backendem opartym o *.NET 8*. *PostgreSQL* oferuje również wsparcie dla zaawansowanych typów danych, takich jak *uuid* czy tablice, które okazały się przydatne przy modelowaniu struktur takich jak listy wersji, zależności czy metadane modów. Istotnym czynnikiem była także wysoka wydajność zapytań — nawet przy pracy z dużymi kolekcjami danych, jak np. statystyki pobrań, oceny czy liczby ulubionych modyfikacji.

2.3 .NET oraz Entity Framework

Platforma *.NET* wraz z *Entity Framework Core* została wybrana ze względu na swoją popularność, stabilność oraz bardzo dobrą integrację z ekosystemem narzędzi firmy *Microsoft*. Umożliwia szybkie tworzenie nowoczesnych, wielowarstwowych aplikacji webowych, jednocześnie oferując wysoką wydajność oraz bezpieczeństwo. Z kolei *Entity Framework Core* pozwala znacząco przyspieszyć rozwój aplikacji dzięki eliminacji potrzeby pisania

dużej ilości zapytań SQL oraz automatycznemu odwzorowaniu struktury bazy danych na modele domenowe. Obie technologie są również intensywnie rozwijane, posiadają długoterminowe wsparcie oraz bogatą dokumentację, co czyni je bezpiecznym i przyszłościowym wyborem dla projektu tej skali.

2.3.1 .NET 8

.NET 8 to najnowsza w momencie rozpoczęcia pracy wersja wieloplatformowej platformy programistycznej firmy Microsoft, wydana w ramach modelu Long-Term Support (LTS). Wprowadza szereg usprawnień wydajnościowych, rozszerzeń językowych oraz ulepszeń związanych z AOT (Ahead-of-Time Compilation), które pozwalają między innymi na szybsze uruchamianie aplikacji i mniejsze zużycie pamięci. Wersja ta pozwala na wykorzystanie nowości języka *C# 12*, takich jak kolejne usprawnienia składni *primary constructors* czy rozszerzenia *using static*. Dodatkowo, *.NET 8* oferuje uproszczony model konfiguracji aplikacji, rozszerzone wsparcie dla *minimal APIs* oraz usprawnienia w zakresie testowania i obserwowalności aplikacji (między innymi wbudowane OpenTelemetry) [dotnet8overview2025] [dotnet8runtime2025]. Dzięki tym możliwościom, aplikacja mogła być rozwijana zgodnie z nowoczesnymi praktykami i była przygotowana na przyszłościową skalowalność.

W projekcie platforma *.NET 8* została wykorzystana do implementacji warstwy backendowej - REST API, uwierzytelnienia, logiki biznesowej oraz integracji z bazą danych.

2.3.2 EF Core 8 oraz Npgsql

Entity Framework Core 8 (EF Core 8) to obiektowo-relacyjny mapper (ORM) opracowany przez firmę Microsoft, służący do pracy z bazą danych w sposób deklaratywny; poprzez definiowanie modeli danych i ich relacji w postaci klas *C#*. *EF Core* pozwala generować i zarządzać strukturą bazy danych poprzez migracje, co znacząco przyspiesza rozwój aplikacji i ułatwia zachowanie spójności danych [ef2025].

W projekcie *EF Core* pełnił kluczową rolę w warstwie dostępu do danych, zapewniając wydajne i typowane zapytania *LINQ* oraz automatyczne śledzenie zmian w encjach [ef2025]. *LINQ* (Language Integrated Query) umożliwia tworzenie zapytań bazodanowych w sposób zintegrowany z językiem *C#*, dzięki czemu można pisać zapytania z pełnym wsparciem IntelliSense, refaktoryzacji i kontroli typów. Takie podejście nie tylko zwiększa czytelność kodu, ale także zmniejsza ryzyko błędów składniowych typowych dla zapytań SQL pisanych ręcznie [linq2025pl]. Dzięki integracji z dostawcą *Npgsql* możliwa była pełna zgodność z *PostgreSQL*, w tym wsparcie dla typów specyficznych jak *uuid* [npgsql2025].

Biblioteka ta pozwalała również korzystać z rozszerzeń *PostgreSQL* takich jak *ILIKE*, pełnotekstowe wyszukiwanie oraz funkcje agregujące. Konfiguracja *Fluent API* oraz atry-

buty w modelach umożliwiały precyzyjne dostosowanie struktury bazy, a jednocześnie zachowanie przejrzystego kodu w warstwie logiki aplikacji.

2.4 Biblioteki backendowe

2.4.1 SpaProxy

W projekcie zastosowałem *SpaProxy*, który działa jako pośrednik przekazujący żądania HTTP z backendu (hostowanego na porcie, na przykład w przypadku lokalnej konfiguracji Candlelight `https://localhost:7275`) do serwera deweloperskiego Angulara (na przykład w przypadku lokalnej konfiguracji Candlelight `http://localhost:4200`). Umożliwia to lokalne uruchamianie aplikacji frontendowej z pełnym wsparciem hot-reload, jednocześnie pozwalając backendowi zachować kontrolę nad routingiem i autoryzacją [aspnetproxypaproxymy2025].

Zastosowanie *SpaProxy* znacząco usprawniło i przyspieszyło pracę deweloperską - nie było konieczne ręczne kopiowanie plików builda frontendu do katalogu `wwwroot` oraz możliwa była równoległa praca nad logiką serwera i interfejsem użytkownika. Proxy to jest transparentne w działaniu i eliminuje problemy związane z CORSem podczas lokalnego rozwoju [spaproxy2025].

2.4.2 Steam OpenID

W celu uwierzytelniania użytkowników za pomocą platformy Steam wykorzystałem protokół *OpenID* w implementacji biblioteki `AspNet.Security.OpenId.Steam` [steamopenid2025]. Pozwala ona na łatwe zintegrowanie logowania kontem Steam, co w kontekście repozytorium modów gier stanowi bardzo intuicyjne i wygodne rozwiązanie dla użytkowników końcowych.

Biblioteka ta obsługuje cały proces przekierowania do Steam, autoryzacji użytkownika oraz zwrócenia unikalnego identyfikatora konta Steam (SteamID), który może być dalej przechowywany w bazie danych i kojarzony z kontem w aplikacji. Była ona więc idealnym dopasowaniem do moich potrzeb.

2.4.3 Swagger i dokumentacja API

W projekcie wykorzystałem bibliotekę `Swashbuckle.AspNetCore`, która integruje framework *Swagger* z aplikacjami opartymi o *ASP.NET Core*. Swagger pozwala na automatyczne generowanie dokumentacji REST API na podstawie atrybutów kontrolerów oraz definicji modeli danych. Wygenerowana dokumentacja dostępna jest w formacie *OpenAPI* [swaggerdocs2025]. Dzięki zastosowaniu Swaggera możliwe było nie tylko wygodne

testowanie endpointów podczas pracy deweloperskiej (przy pomocy interaktywnego interfejsu *Swagger UI*), ale również zwiększenie przejrzystości i łatwości w integracji systemu z potencjalnymi zewnętrznymi usługami. Dokumentacja dostępna jest domyślnie pod ścieżką `/swagger`, a jej struktura aktualizuje się automatycznie wraz z wprowadzaniem zmian w kodzie.

Swagger wspiera dodatkowo opisywanie parametrów, odpowiedzi oraz statusów HTTP za pomocą atrybutów, takich jak `[ProducesResponseType]`, `[FromQuery]`, czy `/// <summary>`. Umożliwia to tworzenie kompletnej, aktualnej i łatwej do utrzymania dokumentacji API bez potrzeby pisania jej ręcznie.

2.5 Typescript i Angular

2.5.1 TypeScript

TypeScript to nadzbiór języka *JavaScript* rozwijany przez firmę *Microsoft*, który wprowadza statyczne typowanie oraz szereg nowoczesnych funkcjonalności znanych z języków takich jak *C#* czy *Java*. Dzięki typom, interfejsom i klasom kod jest łatwiejszy do zrozumienia, utrzymania oraz testowania. Pozwala także wykrywać błędy na etapie kompilacji, zanim jeszcze trafią one do przeglądarki.

W kontekście projektu, *TypeScript* znacząco usprawnił implementację komponentów, serwisów oraz modeli danych w *Angularze*, zapewniając przejrzystość i solidność kodu. Użycie interfejsów do reprezentowania struktur danych takich jak gry, modyfikacje czy użytkownicy umożliwiło wygodną integrację z backendem opartym na *.NET*. Dodatkowo, wykorzystanie konstrukcji takich jak `readonly` podniosło jakość kodu oraz zredukowało potencjalne błędy w trakcie przyszłej pracy zespołowej[typescript2025].

2.5.2 Angular

Angular to jedno z najpopularniejszych frameworków frontendowych, rozwijany przez zespół *Google*. Jest to rozwiązanie typu SPA (Single Page Application), umożliwiające budowę wydajnych, modularnych i łatwo testowalnych aplikacji webowych. *Angular* opiera się na architekturze komponentowej, co znacznie ułatwia zarządzanie złożonym interfejsem użytkownika.

W projekcie używałem *Angulara* do stworzenia interfejsu użytkownika, który umożliwia przeglądanie gier, modyfikacji, logowanie się przez Steam, edycję profilu, a także zarządzanie swoimi ulubionymi modami. Skorzystałem między innymi z takich funkcjonalności frameworka jak:

Angular Routing do implementacji nawigacji pomiędzy widokami.

Reactive Forms do obsługi formularzy edycji profilu oraz przesyłania modyfikacji.

HttpClient do komunikacji z backendowym REST API.

Dependency Injection pozwalający na modularne i testowalne serwisy.

Angular w wersji 18 zaoferował dodatkowe usprawnienia pod kątem wydajności i lazy-loadingu, oraz był najnowszą stabilną (LTS) wersją w momencie rozpoczęcia prac nad projektem [angular2025].

2.6 Znaczące biblioteki frontendowe

2.6.1 RxJS

RxJS (Reactive Extensions for *JavaScript*) to biblioteka programowania reaktywnego używana w *Angularze* do pracy ze strumieniami danych i zdarzeń asynchronicznych. Dzięki niej możliwe jest zarządzanie przepływem danych w sposób deklaratywny, z wykorzystaniem operatorów takich jak `map`, `filter`, `switchMap`, `debounceTime` czy `combineLatest` [rxjs2025].

W projekcie *RxJS* był niezwykle przydatnym narzędziem w takich miejscach jak filtrowanie i wyszukiwanie modów/gier, debounceowanie inputów formularzy oraz subskrypcje danych z API. Dzięki asynchronicznemu podejściu można było znacząco uprościć kod komponentów i uniknąć tak zwanego *callback hell* (zagnieżdżone funkcje zwrotne tworzą trudny do odczytania i utrzymania kod).

2.6.2 Angular Material

Angular Material to oficjalna implementacja *Material Design* dla *Angulara*. Oferuje zestaw gotowych komponentów UI zgodnych z wytycznymi *Google*, takich jak przyciski, formularze, karty, okna dialogowe, listy, paginatory czy elementy nawigacyjne [angularmaterial2025]. W projekcie wykorzystałem *Angular Material* do stworzenia nowoczesnego i spójnego interfejsu użytkownika. Kluczowe komponenty to:

- `MatCard` do prezentacji gier i modów w formie kafelków,

- `MatFormField`, `MatInput`, `MatSelect`, `MatSlideToggle` do obsługi formularzy,

- `MatPaginator` do implementacji paginacji modów i gier,

- `MatSnackBar` do pokazywania komunikatów użytkownikowi,

Zaletą *Angular Material* jest responsywność komponentów oraz ich dobra integracja z tematami kolorystycznymi, co pozwoliło mi na spójny i wygodny UI.

2.7 Linting kodu i zapobieganie code smellowi

W celu zachowania wysokiej jakości kodu źródłowego oraz unikania powstawania tak zwanemu *code smellowi* (czyli wzorców prowadzących do potencjalnych błędów lub utrud-

niających rozwój projektu), a także aby zachować branżowe standardy, w projekcie zastosowałem kilka narzędzi wspierających automatyczną analizę kodu oraz jego formatowanie.

2.7.1 GitHub Actions

Do automatyzacji procesu sprawdzania jakości kodu wykorzystałem mechanizm *GitHub Actions* [githubactions2025]. Pozwala on na definiowanie tak zwanego workflow - zadań uruchamianych automatycznie przy określonych zdarzeniach (na przykład push, pull request). Dzięki temu możliwe było automatyczne uruchamianie analizy lintingu i testów jednostkowych przy każdej zmianie w kodzie, co pomaga w szybkim wykrywaniu błędów oraz zapewnia spójność kodu w całym zespole.

W przypadku Candlelight, automatycznie uruchamiany jest linting frontendu i testy zarówno frontendu jak i backendu przy każdym pushu i pull requestcie.

[githubactions2025]

2.7.2 ESLint

[eslintdocs2025]

2.7.3 Prettier

[prettierdocs2025]

2.7.4 ReSharper

[resharper2025]

2.7.5 SonarQube

[sonarqube2025vs]

2.8 Biblioteki testowe

2.8.1 Karma Jasmine

[karmarunner2025]

2.8.2 NUnit

[nunit2025]

2.8.3 Moq

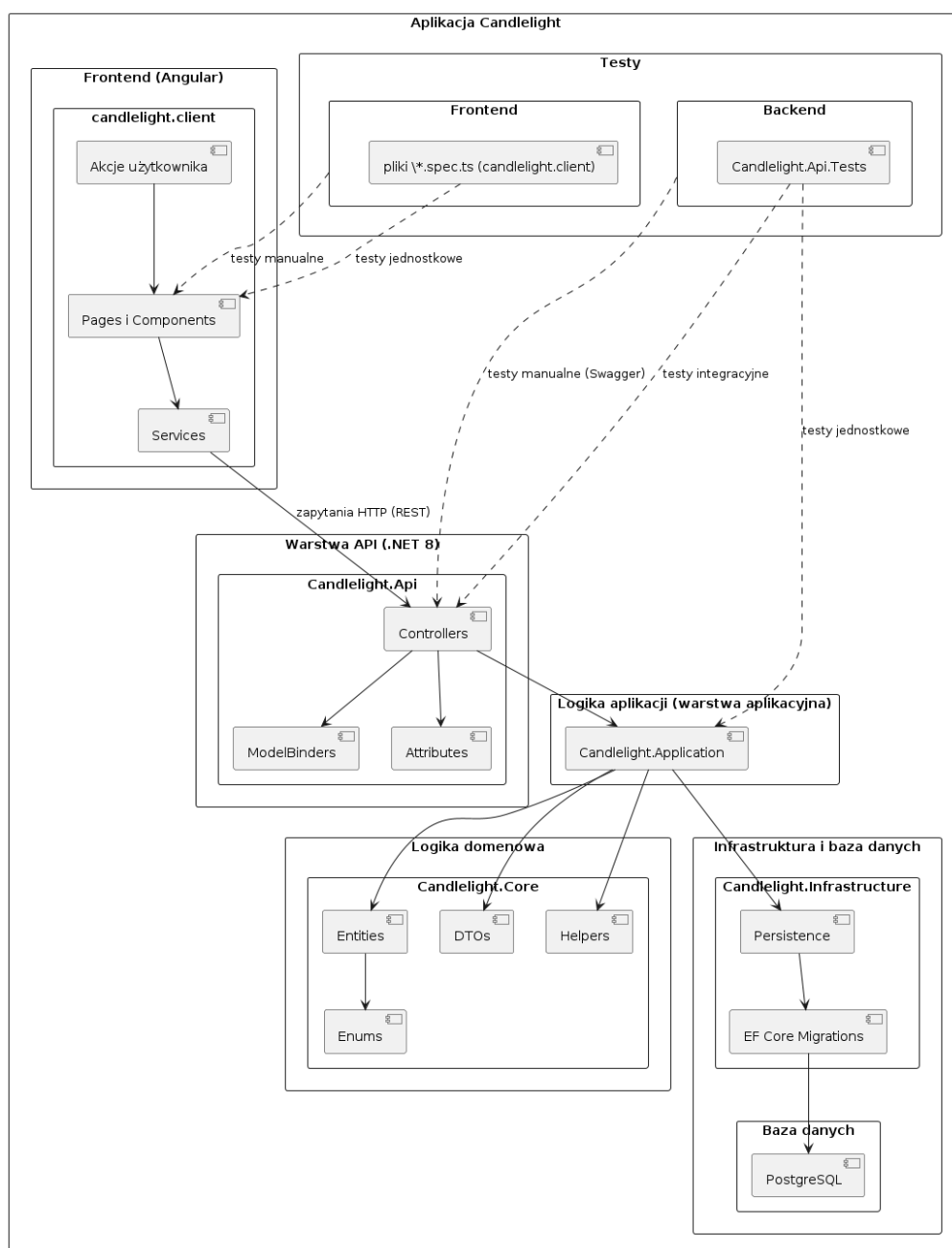
[moqquickstart2025]

2.8.4 Coverlet

//TODO - consider if not remove []

Rozdział 3

Architektura projektu



Rysunek 3.1: Struktura warstwowa aplikacji Candlelight

Struktura projektu została zaprojektowana w duchu architektury wielowarstwowej, umożliwiającej jasny podział odpowiedzialności oraz łatwą skalowalność i utrzymanie kodu. Poniżej przedstawiono główne warstwy i komponenty systemu.

Struktura warstwy API

Projekt `Candlelight.Api` odpowiada za obsługę zapytań HTTP, pełniąc rolę kontrolera dostępowego do logiki aplikacji. Znajdują się tu kontrolery (**Controllers**), atrybuty do walidacji (**Attributes**), customowe **ModelBinders** i pliki statyczne w katalogu `wwwroot`. API aplikacji Candlelight korzysta z architektury REST (Representational State Transfer), co oznacza, że komunikacja z klientem odbywa się za pomocą standardowych metod protokołu HTTP (GET, POST, PUT, DELETE), a zasoby aplikacji są identyfikowane poprzez unikalne URI i reprezentowane w formacie JSON, zapewniając czytelność, skalowalność oraz łatwość integracji z frontendem i innymi usługami. Została ona zaimplementowana przy użyciu frameworka ASP.NET Core, który umożliwia tworzenie kontrolerów, routingu oraz automatyczną serializację i deserializację danych JSON

Struktura warstwy aplikacyjnej

Projekt `Candlelight.Application` zawiera logikę biznesową aplikacji. Moduł **Services** zawiera serwisy odpowiadające za operacje na grach, modach i użytkownikach. Jest to warstwa pośrednia między API a infrastrukturą.

Warstwa dostępu do danych

Projekt `Candlelight.Infrastructure` zawiera implementację dostępu do danych. W katalogu `Persistence/Data` zdefiniowano kontekst bazy danych oraz konfiguracje encji. Folder `Migrations` przechowuje migracje bazy danych generowane przez EF Core 8. Wykorzystywany jest provider `Npgsql` umożliwiający współpracę z PostgreSQL.

Warstwa domenowa

Projekt `Candlelight.Core` gromadzi definicje encji domenowych (**Entities**), transferowych (**Dtos**), typów wyliczeniowych (**Enums**) oraz pomocnicze klasy narzędziowe (**Helpers**). Stanowi centralny punkt współdzielony pomiędzy pozostałymi warstwami aplikacji.

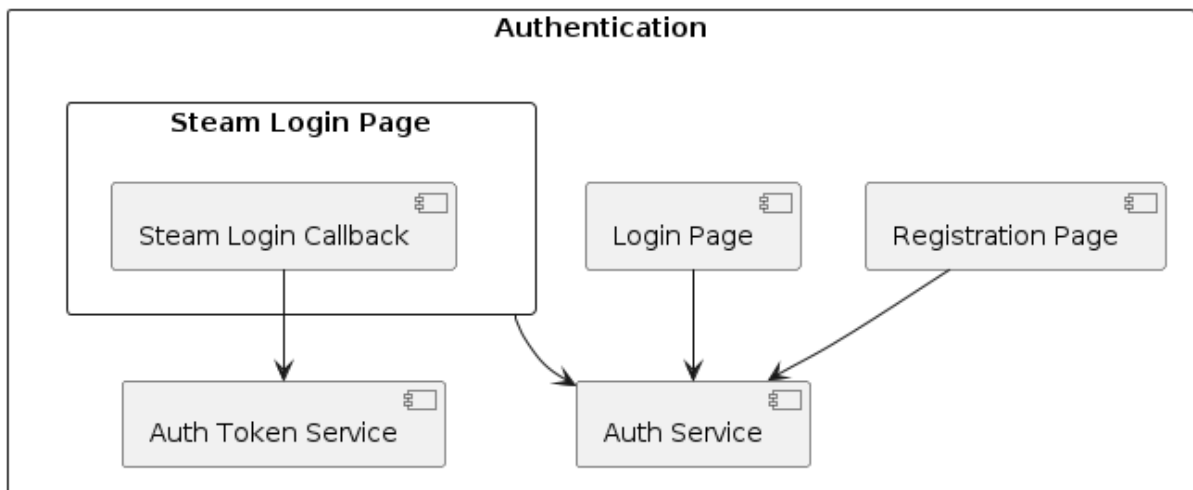
Frontend

Frontend znajduje się w katalogu `candlelight.client` i został zbudowany przy użyciu frameworka *Angular 18*. Główna logika podzielona została na moduły odpowiadające poszczególnym funkcjonalnościom, główne z nich to:

- **auth**: logowanie, rejestracja, Steam login,
- **games**: lista gier, szczegóły gry, dodawanie gier,
- **mods**: przeglądanie, szczegóły, dodawanie modów,
- **user-profile**: profil użytkownika, edycja, ulubione gry i mody,
- **shared**: komponenty wielokrotnego użytku (np. paginator, filtry, guardy),
- **public**: zasoby statyczne takie jak `assets` i `icons`.

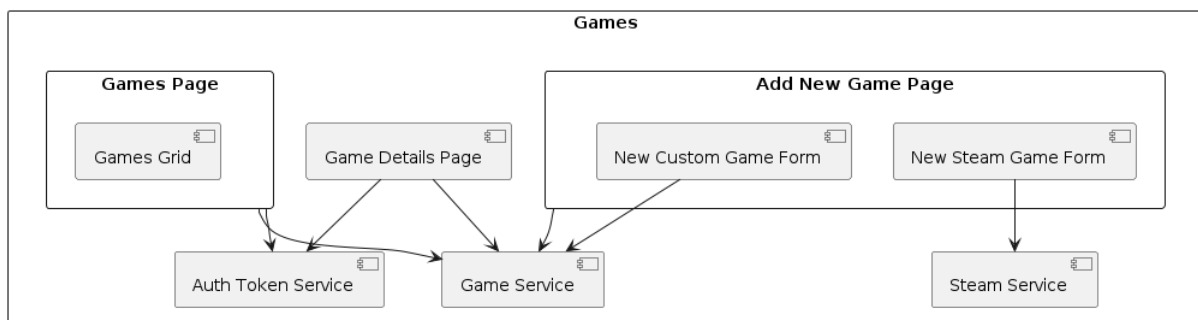
Struktura ta wspiera modularność oraz umożliwia łatwe dodawanie nowych komponentów.

Struktura logiki frontendowej - autentykacja użytkowników



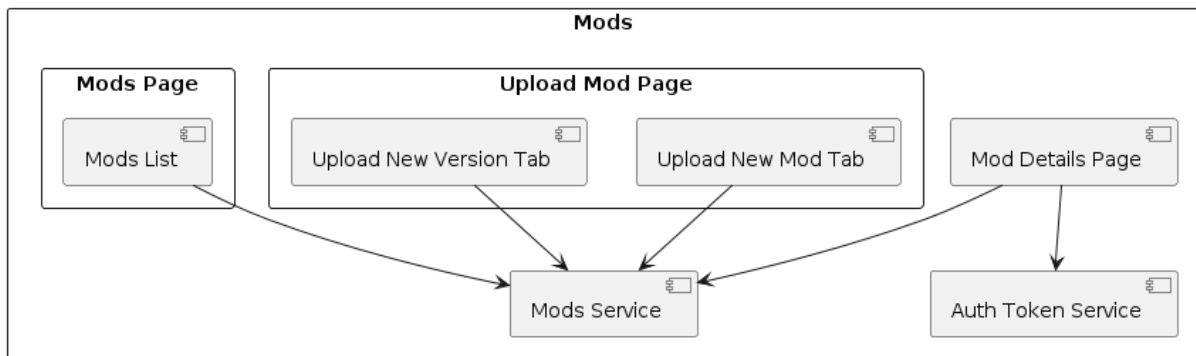
Rysunek 3.2: Diagram struktury logiki frontendowej dotyczącej autentykacji użytkowników

Struktura logiki frontendowej - Games



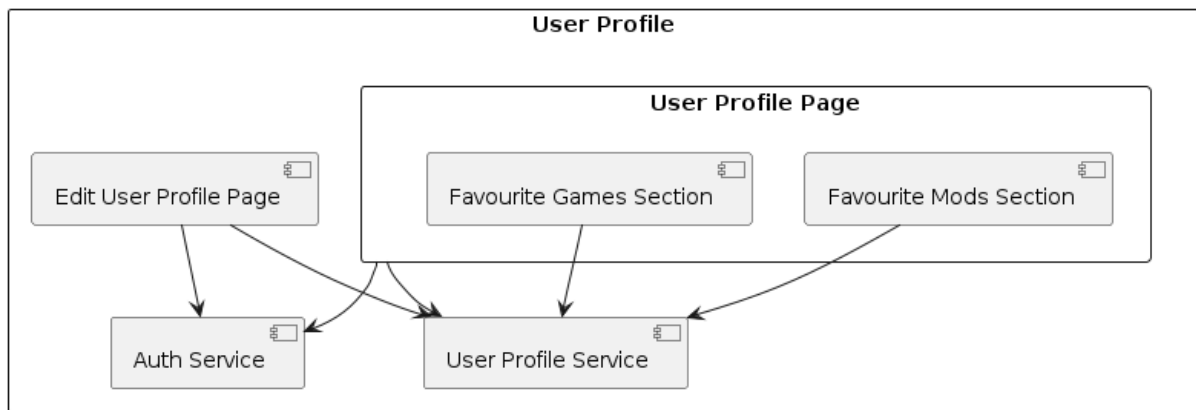
Rysunek 3.3: Diagram struktury logiki frontendowej dotyczącej gier

Struktura logiki frontendowej - Mods



Rysunek 3.4: Diagram struktury logiki frontendowej dotyczącej modyfikacji

Struktura logiki frontendowej - User Profile



Rysunek 3.5: Diagram struktury logiki frontendowej dotyczącej profili użytkowników

Testy jednostkowe

Projekt `Candlelight.Api.Tests` zawiera testy jednostkowe dla backendu. Testy te wykorzystują frameworki `xUnit` i `Moq`, umożliwiając weryfikację poprawności działania serwisów i kontrolerów w izolacji. Dodatkowo podczas pracy nad kodem wykonywałem regularne testy manualne (często przy użyciu *Swaggera*).

Frontend (*`candlelight.client`*) korzysta z biblioteki do testów jednostkowych Karma Jasmine. Testy te są zawarte w plikach `*.spec.ts` i są wydzielone na poszczególne komponenty *Angularowe*.

Zarządzanie konfiguracją i CI/CD

Folder `.github/workflows` zawiera konfiguracje GitHub Actions, które odpowiadają za automatyzację procesu budowania, testowania i wdrażania aplikacji. Dzięki integracji CI/CD (*Continuous Integration / Continuous Deployment*) możliwe jest szybkie wychwytywanie błędów i zapewnienie spójnej jakości kodu.

Zasoby statyczne backendu

W katalogu `wwwroot` znajdują się zasoby statyczne służące do przechowywania przesyłanych przez użytkowników plików, takich jak awatary, okładki gier i obrazy modów. Pliki są porządkowane w foldery zgodnie z identyfikatorami encji, co ułatwia zarządzanie oraz zapewnia unikalność nazw.

Rozdział 4

Implementacja funkcjonalności aplikacji

4.1 Obsługa autoryzacji użytkownika

4.1.1 Endpointy związane z autoryzacją użytkownika

UserAccess			^
OPTIONS	/api/UserAccess/SendRegistrationForm	Preflight.	▼
POST	/api/UserAccess/SendRegistrationForm	Registers a user with a Candlelight account.	▼
OPTIONS	/api/UserAccess/SendLoginForm	Preflight.	▼
POST	/api/UserAccess/SendLoginForm	Logs in the user with their Candlelight account.	▼
OPTIONS	/api/UserAccess/GetUsersList	Preflight.	▼
GET	/api/UserAccess/GetUsersList	Returns all users with their info.	▼
GET	/api/UserAccess/LinkSteam	Used to link Candlelight account to Steam account.	▼
GET	/api/UserAccess/LinkSteamCallback	Callback after linking Candlelight account with Steam.	▼
GET	/api/UserAccess/CurrentUser	Returns info about currently logged-in user.	▼
GET	/api/UserAccess/DebugToken	Used for debugging JWT.	▼
GET	/api/UserAccess/SteamLogin	Used to log in using Steam.	▼
GET	/api/UserAccess/SteamCallback	Callback used when logging in by Steam. Do not use manually unless debugging.	▼
GET	/api/UserAccess/GetCurrentUserId	Returns the ID of the currently logged-in user.	▼
POST	/api/UserAccess/Logout	Logs out the currently authenticated Steam user (cookie-based). JWT users are handled on the frontend.	▼

Rysunek 4.1: Sekcja w Swaggerze pokazująca endpointy w kontrolerze UserAccess

4.1.2 Rejestracja przy pomocy formularza

Listing 4.1: Kontroler UserAccessController: endpoint SendRegisterForm

```
1  /// <summary>
2  /// Registers a user with a Candlelight account.
3  /// </summary>
```

```

4      [HttpPost]
5      [ActionName("SendRegisterForm")]
6      [Route("SendRegistrationForm")]
7      public async Task<ActionResult> PostAsync([FromBody]
      RegistrationForm form)
8      {
9          try
10         {
11             if (!AuthenticationService.IsRegistrationFormValid(
12                 form))
13             {
14                 return BadRequest("Data_model_is_invalid.");
15             }
16
17             var user = await _authenticationService.RegisterUser
18                 (form);
19             return Ok(new { id = user.Id });
20         }
21         catch (Exception ex)
22         {
23             return StatusCode(500, new { error = "Internal_
24                 Server_Error", details = ex.Message });
25         }
26     }

```

Endpoint `SendRegisterForm` ?? realizuje rejestrację użytkownika przy pomocy wypełnionego przez użytkownika formularza do rejestracji. Formularz ten składa się z 4 elementów typu `string` - `UserName` (nazwa użytkownika), `Email`, `Password` i `ConfirmPassword` (hasło oraz jego kopia, która powinna być identyczna). Walidacja polega na sprawdzeniu, czy:

- email jest poprawny (wykorzystywany jest do tego `EmailHelper`, który korzysta z `EmailAddressAttribute` zaczerpnięty z `System.ComponentModel.DataAnnotations`)
- hasło nie jest za krótkie oraz czy jest one identyczne jak `string ConfirmPassword`
- nazwa użytkownika jest między sześcioma a trzydziestoma znakami
- żaden z powyższych `stringów` nie jest pusty.

4.1.3 Logowanie przy pomocy emaila oraz hasła

Listing 4.2: Kontroler UserAccessController: endpoint SendLoginForm

```
1  /// <summary>
2  /// Logs in the user with their Candlelight account.
3  /// </summary>
4  [HttpPost]
5  [Route("SendLoginForm")]
6  [ActionName("SendLoginForm")]
7  public async Task<ActionResult> PostAsync([FromBody]
   LoginForm form)
8  {
9      try
10     {
11         if (!AuthenticationService.IsLoginFormValid(form))
12         {
13             return BadRequest("Data_model_is_invalid.");
14         }
15         var result = await _authenticationService.
            AttemptLogin(form);
16         if (result == null)
17         {
18             return Unauthorized(form);
19         }
20         var token = _authenticationService.GenerateJwtToken(
            result);
21         return Ok(new { Token = token });
22     }
23     catch (Exception ex)
24     {
25         return StatusCode(500, new { error = "Internal_
            Server_Error", details = ex.Message });
26     }
27 }
```

Listing 4.3: Serwis AuthenticationService: metoda GenerateJwtToken

```
1  public string GenerateJwtToken(AppUser user)
2  {
3      var key = new SymmetricSecurityKey(Encoding.UTF8.
        GetBytes(_jwtKey));
4      var credentials = new SigningCredentials(key,
```

```

SecurityAlgorithms.HmacSha256);
5
6    var claims = new[]
7    {
8        new Claim(JwtRegisteredClaimNames.Sub, user.Id.
          ToString()),
9        new Claim(JwtRegisteredClaimNames.Email, user.Email
          !),
10       new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid
          ().ToString())
11    };
12
13    var token = new JwtSecurityToken(
14        _issuer,
15        _audience,
16        claims,
17        expires: DateTime.UtcNow.AddHours(2),
18        signingCredentials: credentials
19    );
20
21    return new JwtSecurityTokenHandler().WriteToken(token);
22 }

```

Powyższy fragment kodu ?? przedstawia implementację endpointu logowania użytkownika do aplikacji Candlelight za pomocą emaila oraz hasła. Korzysta on z serwisu AuthenticationService aby zweryfikować przesłane dane a następnie zalogować użytkownika i zwrócić token JWT. W aplikacji Candlelight token JWT jest używany do uwierzytelniania użytkownika przy każdym kolejnym żądaniu do zabezpieczonych zasobów API, zapewniając tym samym mechanizm sesji bezstanowej oraz kontrolę dostępu do danych zgodnie z tożsamością zalogowanego użytkownika

4.2 Zarządzanie profilem użytkownika

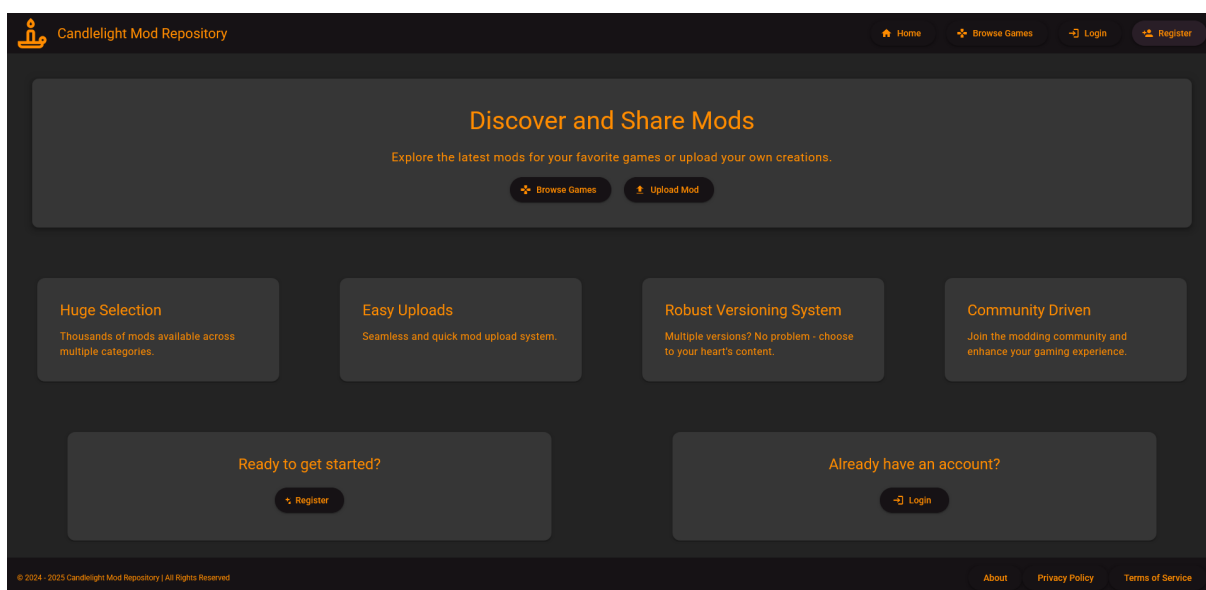
4.3 Funkcjonalności związane z grami

4.4 Funkcjonalności związane z modyfikacjami

4.4.1 Przesyłanie modyfikacji oraz ich wersji

4.4.2 Lista modyfikacji dla gry

4.5 Strona główna oraz UX aplikacji



4.6 Integracje z zewnętrznymi serwisami

4.6.1 Steam

Rejestracja oraz logowanie za pomocą konta Steam

4.6.2 Google Fonts

Interfejs użytkownika aplikacji korzysta z biblioteki Angular Material [[angularmaterial2025](#)], która domyślnie integruje się z usługą Google Fonts (CDN Google Fonts `fonts.googleapis.com`, `fonts.gstatic.com`) w celu ładowania czcionki Roboto [[roboto2025](#)] oraz zestawu ikon Material Icons [[materialicons2025](#)]. Dzięki temu możliwe było uzyskanie spójnego i nowoczesnego wyglądu aplikacji przy minimalnym nakładzie konfiguracji.

Alternatywą dla ładowania czcionek i ikon z Google Fonts jest ich lokalne hostowanie w katalogu statycznym aplikacji oraz ręczne dołączenie ich za pomocą reguł `@font-face` w plikach stylów. Takie rozwiązanie pozwala uniknąć zależności od zewnętrznych serwerów oraz może być wymagane w środowiskach z ograniczonym dostępem do Internetu (intranety, sieci zamknięte).

W przypadku projektu Candlelight nie było konieczności stosowania lokalnej wersji, ponieważ aplikacja nie przetwarza danych szczególnie wrażliwych na poziomie frontendowym, środowisko produkcyjne nie ogranicza połączeń z Internetem, a korzystanie z CDN Google Fonts zapewnia lepszą wydajność dzięki globalnej dystrybucji oraz umożliwia cacheowanie czcionek w przeglądarce użytkownika, co przekłada się na szybsze ładowanie interfejsu i mniejsze zużycie zasobów.

Dodatkowo, użycie domyślnych linków do czcionek i ikon rekomendowanych przez Angular Material zapewnia pełną kompatybilność z komponentami biblioteki bez potrzeby ręcznej konfiguracji i dostosowywania stylów, co oszczędziło niepotrzebnej pracy i straty czasu.

Rozdział 5

Podsumowanie projektu

5.1 Trudności i wyzwania

5.2 Wnioski z projektu

5.3 Potencjalne kierunki rozwoju projektu

... wydzielenie mikroservisów komunikujących się przy użyciu Kafki oraz konteneryzacja, integracja z VirusTotal, QoL features, UX work, hosting ...

